

Comparing SVD, Neural Collaborative Filtering, and Bayesian PMF for Unseen Movie Rating Prediction

Introduction and Literature Review

A central task in recommender systems is estimating how a user would rate an item that they have not yet seen or rated. In the movie domain, this problem is especially important because accurate rating prediction allows recommendation platforms to move beyond general popularity and infer more personalized user preferences. We restrict all three models to the same explicit rating prediction setting using only user and movie identifiers. In this project, we study explicit movie rating prediction on the MovieLens dataset by comparing three machine learning approaches: SVD based matrix factorization, Neural Collaborative Filtering, and Bayesian Probabilistic Matrix Factorization. These models were selected because they represent three distinct ways of modeling user item interactions, namely classical latent factor modeling, neural nonlinear modeling, and probabilistic latent factor modeling.

Early work in recommender systems was dominated by neighborhood based collaborative filtering. Resnick et al. introduced GroupLens, one of the first collaborative filtering systems, which predicted ratings by computing weighted averages of scores from similar users using Pearson correlation [1]. This work established the basic idea that user preferences could be inferred from patterns of agreement among users. However, neighborhood based approaches are highly sensitive to sparsity and become less effective as user item matrices grow larger and more incomplete. These limitations motivated later methods that could model hidden structure more directly.

A major step forward came from explicitly modeling the systematic biases present in rating data. Bell and Koren showed that a simple baseline model built from a global mean, a user bias, and an item bias can outperform many more complicated neighborhood methods [2]. This insight remains important because it separates stable rating tendencies from user item interaction effects and provides a strong starting point for more advanced models. In our project, these bias components are incorporated directly into the SVD based matrix factorization model.

Building on this baseline idea, Koren et al. demonstrated that matrix factorization can significantly improve recommendation accuracy by decomposing the user item matrix into low dimensional latent factors [3]. In this framework, each user and each movie is represented by a vector in a shared latent space, and ratings are predicted through interactions between these vectors together with bias terms. Matrix factorization is particularly effective for sparse recommendation data because it captures hidden preference structure instead of relying only on direct overlap between users. This work provides the theoretical foundation for our first model, which implements matrix factorization manually using NumPy.

Although matrix factorization is highly effective, its standard formulation models user item interaction through a linear inner product. He et al. argued that this assumption may be too restrictive to capture more complex patterns in user behavior [4]. Their Neural Collaborative Filtering framework replaces the fixed linear interaction with a multilayer perceptron that learns a nonlinear matching function between user and item representations. Their experiments on MovieLens showed that neural modeling can improve performance over traditional matrix factorization under suitable settings. This motivates our second model, which uses learned user and movie embeddings together with a multilayer perceptron to model more flexible user item relationships.

A different extension of matrix factorization comes from probabilistic modeling. Salakhutdinov and Mnih proposed Probabilistic Matrix Factorization, which treats user and item latent vectors within a probabilistic framework rather than as fixed point estimates [5]. Their work showed that probabilistic formulations can generalize well on sparse data and provide a natural basis for uncertainty aware interpretation. This idea is important for our project because it motivates the Bayesian model in our comparison. While the three models in our study all address the same rating prediction task, the Bayesian formulation differs in that it emphasizes uncertainty and distributional reasoning in addition to predictive accuracy.

Taken together, these studies describe a clear historical progression in rating prediction research. Early neighborhood methods established the collaborative filtering problem, bias models and matrix factorization improved scalability and predictive power, neural approaches relaxed the linear interaction assumption, and Bayesian approaches introduced probabilistic interpretation. Our project compares these three perspectives on the same MovieLens prediction task in order to examine how different modeling assumptions influence performance under a shared experimental setting. This comparison also provides the basis for the methodological decisions described in the next section, including our shared data split, common evaluation metric, and model specific implementation choices.

ML Methodology and Data Collection

This project studies the explicit rating prediction task in recommender systems, which aims to predict how users would rate movies they have not yet rated based on historical rating records. To ensure that the problem is representative and comparable with existing work, we use the MovieLens dataset as the core data source. MovieLens is one of the most widely used benchmark datasets in recommender systems research, containing a large number of anonymous explicit movie ratings. Its clear structure and broad use in prior literature make it suitable for studying user preference modeling and rating prediction. In this project, we focus specifically on explicit movie rating prediction rather than click-through rate prediction, ranking, or implicit feedback recommendation. This setting allows us to directly compare how different modeling assumptions affect predictive performance.

In order to ensure comparability across models, we adopt a unified data processing and evaluation framework. All three models use the same MovieLens data and define the task as rating prediction based only on `user_id` and `movie_id`. No additional user attributes, movie metadata, or textual features are introduced. We use the shared pre-saved train, validation, and test split prepared by the group, with an 80 10 10 ratio and random seed 42. All ratings remain on the original one to five scale, and final predictions are clipped to the same range. We use RMSE as the primary evaluation metric because it directly measures the average prediction error for explicit ratings and is more appropriate for this task than ranking based or click based measures.

The first method is SVD based matrix factorization. This model represents each user and each movie with a low dimensional latent vector and predicts ratings through a global mean, user bias, movie bias, and latent factor interactions. Compared with earlier weighted average methods based on similar users, matrix factorization can extract hidden preference structure more effectively from sparse rating data, making it a strong classical baseline in recommender systems research. We retain the core components of bias terms and latent factor interactions, and train the model using stochastic gradient descent with regularization. In addition to serving as a baseline, this model also satisfies the project requirement of manual implementation, since we directly implemented parameter updates, bias modeling, and latent factor learning rather than relying on high level recommendation libraries.

The second method is Neural Collaborative Filtering. Its main idea is to replace the linear interaction used in traditional matrix factorization with a neural network. We learn user and movie embeddings separately and pass them through a multilayer perceptron to model more complex nonlinear user item relationships. Compared with the fixed inner product in matrix factorization, Neural CF provides a more flexible way to model rating patterns. In this project, we developed and tested multiple versions of the Neural CF model. The earlier versions used smaller embedding

dimensions and simpler hidden layers as baseline settings. We then gradually increased model size, and in the final version we added a deeper network and dropout to improve both model capacity and generalization. The final version performed better than the earlier ones on both training and validation data, suggesting that Neural CF is sensitive to architecture design and hyperparameter choices. While the course introduces the general idea of multilayer perceptrons, our project extends that idea to recommendation by combining user and movie embeddings in a neural model.

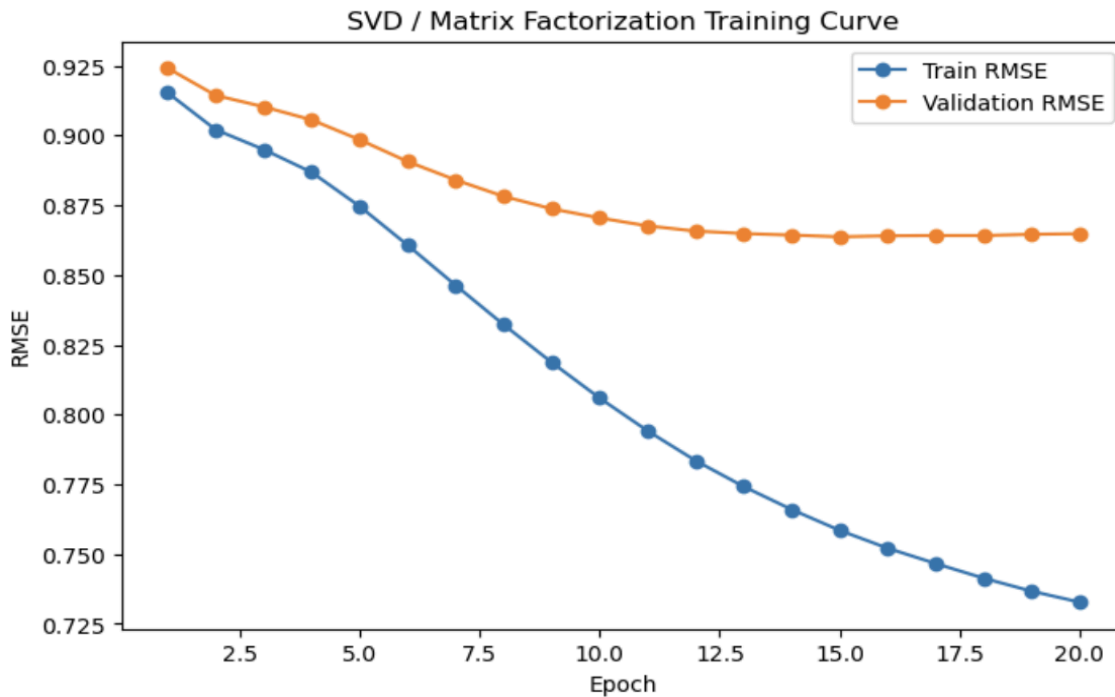
The third method is Bayesian Probabilistic Matrix Factorization. Based on matrix factorization, this model introduces probabilistic modeling into the learning process. Rather than treating user and movie latent vectors as fixed parameters, it represents them through probability distributions. This gives the model a richer interpretation than a standard point estimate approach, since it can describe uncertainty in predicted ratings as well as the predictions themselves. In sparse recommendation settings, this probabilistic perspective can provide a more informative view of user item interactions. In our project, Bayesian PMF follows the same core setup as the other two models by using only `user_id` and `movie_id` and by being evaluated under the same shared data split and RMSE metric. According to our results, Bayesian PMF performs very close to SVD in predictive accuracy while also providing additional information such as rating probability distributions. This makes it valuable not only as a competitive predictive model, but also as a probabilistic alternative to the two point estimate methods.

Although this project is fundamentally based on collaborative filtering, matrix factorization, and neural networks covered in the course, we made several extensions beyond the most basic classroom setting. First, we unified all three methods within the same explicit rating prediction framework and evaluated them under the same data split, input setting, and RMSE metric. Second, for Neural CF, we carried out multi version tuning by adjusting embedding dimensions, hidden layer structure, and regularization. Third, for Bayesian PMF, we emphasized not only final RMSE but also uncertainty related outputs such as rating distributions. In addition, we developed a simple interactive dashboard to display parameter settings, prediction results, and example rating distributions for the Bayesian model. While this dashboard is not part of the core training pipeline, it provides an intuitive way to inspect model behavior and support result interpretation.

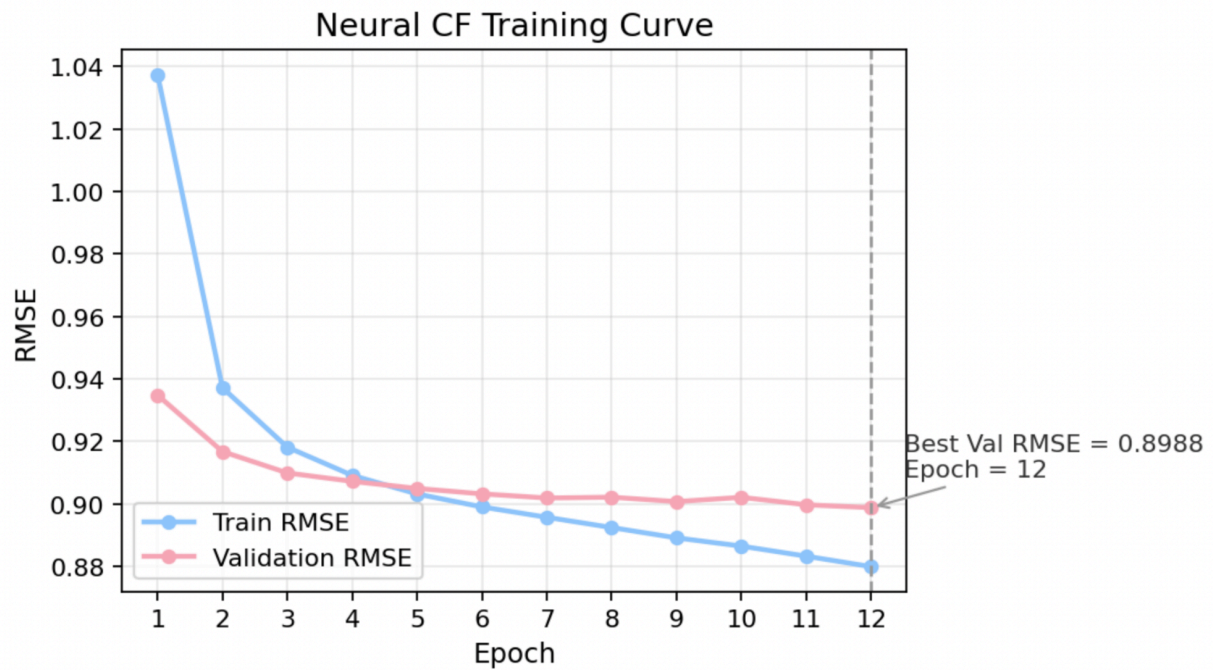
ML Results and Interpretation

We examined the performance of all three models in explicit movie rating prediction with the 80 10 10 train-validation/test data split that we have used for all experiments. To enable fair comparison, we applied each model only to `user_id` and `movie_id` features and obtained predictions on the range from one to five stars. Performance metrics are Root Mean Squared Error (RMSE), which allows us to observe significant differences between approaches on the same task.

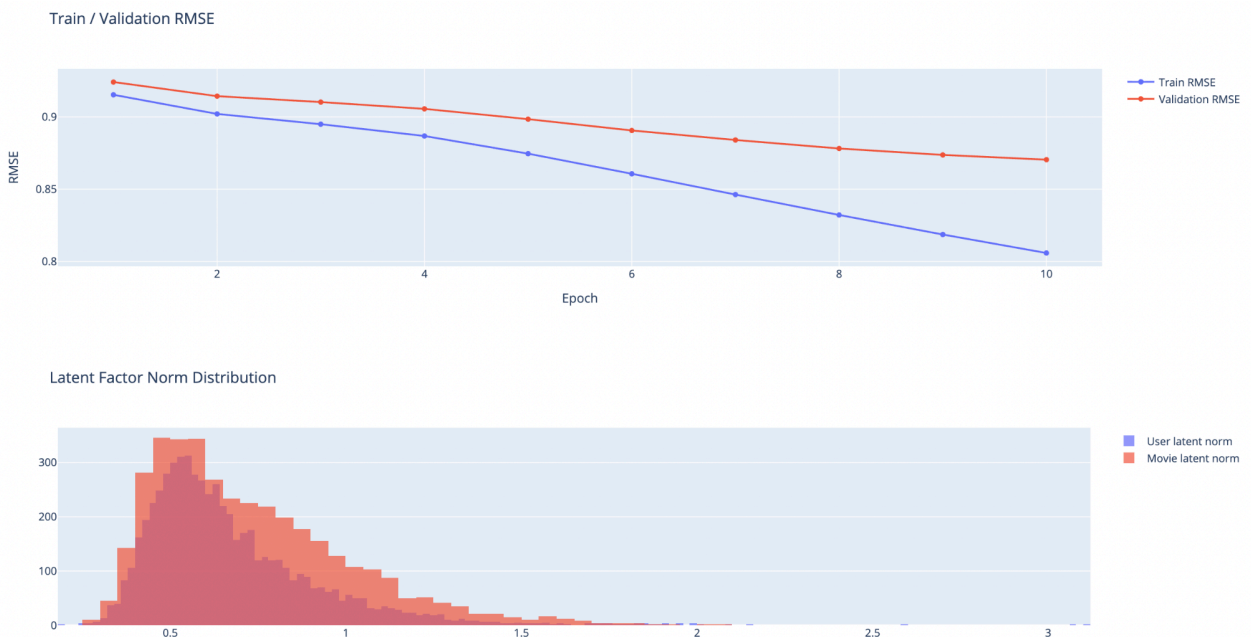
It is clear that among our implementations, Matrix Factorization (SVD) performed best at predicting explicit movie ratings. Its validation RMSE is 0.8648 and the corresponding test RMSE equals 0.8612. One can say that in this case the classic latent factor model proved very effective at modeling user-movie interactions in this task. Since SVD represents each user and each movie in the dataset as low-dimensional latent vectors in addition to learning global, user-level and movie-level bias terms, it captures the key signals without overfitting. Its training curve demonstrates constant improvements during each epoch.



Neural Collaborative Filtering (Neural CF) yielded the highest validation RMSE 0.8988 and test RMSE 0.8922, being the weakest of our models. The current results suggest that under our project limitations the increase of flexibility brought by neural networks did not lead to any improvement in model's generalization. Indeed, by replacing linear interactions in the matrix factorization with the use of multilayer perceptrons, Neural CF can theoretically learn more complex patterns of user-movie interaction. However, the use of two inputs only – `user_id` and `movie_id` – did not seem to be enough to achieve that potential since such models tend to suffer from the curse of dimensionality. Although we improved performance of Neural CF in comparison with previous iterations, its RMSE remained relatively high.



Bayesian PMF yielded the second best results with validation RMSE 0.8704 and test RMSE 0.8654. Even though the prediction error of this approach is somewhat higher than of SVD's, it can be seen as competitive. As expected from Bayesian PMF, in addition to learning the parameters of the latent factors of users and movies it introduces some additional flexibility in the form of probabilistic assumptions. Thus, our implementation of the approach gave us not just point predictions but also uncertainty-aware interpretations of rating outcomes. Therefore, Bayesian PMF cannot be considered worse than SVD in terms of RMSE error but it provides probabilistic interpretation that is not inherent in classic matrix factorization.



In summary, the comparative evaluation showed that SVD performs best of all, followed by Bayesian PMF, which in turn yields better results than Neural CF. Such a difference in

performances suggests that for standard MovieLens-style explicit rating problems in combination with a highly limited set of features the classical latent factor models prove to be very powerful baselines. In particular, these results suggest that the model's simplicity as well as compatibility with the data structure may play a larger role than flexibility.

At the same time, the results correlate nicely with the ability of each model to capture user-movie relationships. Thus, the fact that SVD was best proves that in this dataset user preferences and movie characteristics can be efficiently represented with the use of latent factors. Bayesian PMF works similarly but adds additional flexibility, thus explaining its performance. Finally, Neural CF's relatively poor performance can be explained by lack of input features. The key finding of our project in terms of empirical results is that among the three models SVD performs best with both smallest validation and test RMSE values. At the same time, Bayesian PMF offers competitive performance with a nice feature of uncertainty-aware rating outcomes predictions. Finally, Neural CF demonstrates how increased complexity in the form of flexible nonlinearities does not necessarily help to improve prediction.

Conclusions and Future Work

For our recommendation models, we compared three models based on explicit movie ratings: Matrix Factorization (SVD), Bayesian Probabilistic Matrix Factorization, and Neural Collaborative Filtering. To ensure comparability between methods, all three models were trained and tested under identical circumstances using the exact same data split, user_id-movie_id core inputs, and root mean squared error metric. Our main finding is that SVD yielded the best results with RMSE scores of 0.8648 (validation) and 0.8612 (test) compared to 0.8654 for test performance of Bayesian PMF, and 0.8922 for test performance of Neural CF. The latter two showed some promising capabilities for richer prediction, such as uncertainty awareness in the case of Bayesian PMF, but failed to surpass the classical latent factor model in accuracy on a single input setting.

The assumptions about the data used to train and evaluate our recommendations are valid for the problem statement. Namely, all three models we chose are designed specifically for user-item recommendation problems and we trained and tested them in identical fashion without any deviation. However, one limitation of this approach is obvious. The input set that we used included solely user_id and movie_id values, so none of our models was allowed to leverage richer information, such as user or item metadata from the original files. It is possible that more expressive input sets could have improved the performance of the models, particularly for Neural CF. As another limitation, we would like to mention that our experiment concentrated on RMSE-based comparison only. We did not attempt to introduce any other user/item metadata into

our recommendation task as it would have been possible based on the available data. It is possible that some other input features would allow us to further increase model performances. Moreover, even though Bayesian PMF provided a probabilistic interpretation of our predictions, the main criteria for performance comparison was still RMSE. If we were to pursue our research into recommendation, there are several directions to consider. First, we might want to expand our user and movie features by adding such attributes as age, gender, occupation, and other characteristics from our dataset. Second, we could further tune the Neural CF model as its current performance leaves something to be desired for us. Third, we could experiment with more advanced Bayesian inference procedures, or even try combining side features with latent factors. Lastly, the dashboard could be improved to facilitate results visualization and performance testing by users.

References

- [1] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "GroupLens: An open architecture for collaborative filtering of Netnews," in *Proc. ACM CSCW*, 1994, pp. 175–186.
- [2] R. M. Bell and Y. Koren, "Lessons from the Netflix Prize Challenge," *ACM SIGKDD Explorations Newsletter*, vol. 9, no. 2, pp. 75–79, 2007.
- [3] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [4] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proc. WWW*, 2017, pp. 173–182.
- [5] R. Salakhutdinov and A. Mnih, "Probabilistic matrix factorization," in *Advances in Neural Information Processing Systems*, vol. 20, 2008, pp. 1257–1264.

Groupwork Finalized GitHub Repo link: https://github.com/TinaChen03/DS4420_project

Appendix: Implementation

SVD Based Matrix Factorization in Python

The following code contains the training implementation of our SVD based matrix factorization model. This model was implemented manually in NumPy using stochastic gradient descent.

```
import random
import numpy as np
import pandas as pd
from numba import njit

# Global config
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

TRAIN_PATH = "train_ratings.csv"
VAL_PATH = "val_ratings.csv"
TEST_PATH = "test_ratings.csv"

# Load data
train_df = pd.read_csv(TRAIN_PATH)
val_df = pd.read_csv(VAL_PATH)
test_df = pd.read_csv(TEST_PATH)

# Build mappings
all_user_ids = pd.Index(
    pd.concat([train_df["user_id"], val_df["user_id"],
test_df["user_id"]]).unique()
)
all_movie_ids = pd.Index(
    pd.concat([train_df["movie_id"], val_df["movie_id"],
test_df["movie_id"]]).unique()
)

user2idx = {uid: i for i, uid in enumerate(all_user_ids)}
movie2idx = {mid: i for i, mid in enumerate(all_movie_ids)}

n_users = len(user2idx)
n_items = len(movie2idx)

# Map ids to internal indices
def map_df(df):
    out = df.copy()
    out["u_idx"] = out["user_id"].map(user2idx)
    out["i_idx"] = out["movie_id"].map(movie2idx)
    return out

train_df = map_df(train_df)
val_df = map_df(val_df)
test_df = map_df(test_df)

# Convert to numpy arrays
def df_to_arrays(df):
```

```

    return (
        df["u_idx"].values.astype(np.int64),
        df["i_idx"].values.astype(np.int64),
        df["rating"].values.astype(np.float64)
    )

u_train, i_train, y_train = df_to_arrays(train_df)
u_val, i_val, y_val = df_to_arrays(val_df)
u_test, i_test, y_test = df_to_arrays(test_df)

# SVD / Matrix Factorization core
@njit
def predict_one_svd(u, i, global_mean, bu, bi, P, Q):
    pred = global_mean + bu[u] + bi[i]

    dot = 0.0
    for f in range(P.shape[1]):
        dot += P[u, f] * Q[i, f]
    pred += dot

    if pred < 1.0:
        pred = 1.0
    elif pred > 5.0:
        pred = 5.0

    return pred

@njit
def rmse_score_svd(u_idx, i_idx, y, global_mean, bu, bi, P, Q):
    se = 0.0
    n = len(y)

    for t in range(n):
        pred = predict_one_svd(u_idx[t], i_idx[t], y[t] * 0 + global_mean,
                                bu, bi, P, Q)
        err = y[t] - pred
        se += err * err

    return np.sqrt(se / n)

@njit
def train_svd(
    u_train, i_train, y_train,
    u_val, i_val, y_val,
    n_users, n_items,
    k, epochs, lr, reg, seed
):
    np.random.seed(seed)

    global_mean = np.mean(y_train)

    P = 0.1 * np.random.randn(n_users, k)
    Q = 0.1 * np.random.randn(n_items, k)

    bu = np.zeros(n_users)
    bi = np.zeros(n_items)

```

```

history = np.zeros((epochs, 3))
n = len(y_train)
order = np.arange(n)

for ep in range(epochs):
    np.random.shuffle(order)

    for s in range(n):
        idx = order[s]
        u = u_train[idx]
        i = i_train[idx]
        r = y_train[idx]

        pred = global_mean + bu[u] + bi[i]
        dot = 0.0
        for f in range(k):
            dot += P[u, f] * Q[i, f]
        pred += dot

        err = r - pred

        bu[u] += lr * (err - reg * bu[u])
        bi[i] += lr * (err - reg * bi[i])

        for f in range(k):
            pu = P[u, f]
            qi = Q[i, f]

            P[u, f] += lr * (err * qi - reg * pu)
            Q[i, f] += lr * (err * pu - reg * qi)

    train_rmse = rmse_score_svd(u_train, i_train, y_train, global_mean,
bu, bi, P, Q)
    val_rmse = rmse_score_svd(u_val, i_val, y_val, global_mean, bu, bi,
P, Q)

    history[ep, 0] = ep + 1
    history[ep, 1] = train_rmse
    history[ep, 2] = val_rmse

return global_mean, bu, bi, P, Q, history

def fit_svd_model(k=20, epochs=20, lr=0.01, reg=0.02, seed=42):
    global_mean, bu, bi, P, Q, history = train_svd(
        u_train, i_train, y_train,
        u_val, i_val, y_val,
        n_users, n_items,
        k=k, epochs=epochs, lr=lr, reg=reg, seed=seed
    )

    test_rmse = rmse_score_svd(u_test, i_test, y_test, global_mean, bu, bi,
P, Q)

    model = {
        "global_mean": global_mean,
        "bu": bu,
        "bi": bi,
        "P": P,

```

```

        "Q": Q,
        "history": history,
        "test_rmse": test_rmse,
        "k": k,
        "epochs": epochs,
        "lr": lr,
        "reg": reg,
        "seed": seed
    }
    return model

model_svd = fit_svd_model(
    k=20,
    epochs=20,
    lr=0.01,
    reg=0.02,
    seed=42
)

```

Neural Collaborative Filtering in Python

The following code contains the training implementation of the final Neural CF model used in our comparison.

```

import gc
import copy
import random
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

SEED = 42
DEVICE = torch.device("cpu")

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

set_seed(SEED)

TRAIN_PATH = "train_ratings.csv"
VAL_PATH = "val_ratings.csv"
TEST_PATH = "test_ratings.csv"

DEBUG_SAMPLE = False

EMBED_DIM = 32
HIDDEN_UNITS = (64, 32)
DROPOUT_RATE = 0.1

BATCH_SIZE = 64

```

```

EPOCHS = 12
LEARNING_RATE = 1e-3
PATIENCE = 3

def load_split(train_path=TRAIN_PATH, val_path=VAL_PATH,
test_path=TEST_PATH):
    train_df = pd.read_csv(train_path)
    val_df = pd.read_csv(val_path)
    test_df = pd.read_csv(test_path)

    expected_cols = {"user_id", "movie_id", "rating"}
    for name, df in [("train", train_df), ("val", val_df), ("test",
test_df)]:
        missing = expected_cols - set(df.columns)
        if missing:
            raise ValueError(f"{name} split is missing columns: {missing}")

    return train_df, val_df, test_df

train_df, val_df, test_df = load_split()

def filter_seen_entities(train_df, val_df, test_df):
    train_users = set(train_df["user_id"].unique())
    train_movies = set(train_df["movie_id"].unique())

    val_df = val_df[
        val_df["user_id"].isin(train_users) &
val_df["movie_id"].isin(train_movies)
    ].reset_index(drop=True)

    test_df = test_df[
        test_df["user_id"].isin(train_users) &
test_df["movie_id"].isin(train_movies)
    ].reset_index(drop=True)

    return train_df.reset_index(drop=True), val_df, test_df

train_df, val_df, test_df = filter_seen_entities(train_df, val_df, test_df)

def build_id_maps(train_df):
    user_ids = np.sort(train_df["user_id"].unique())
    movie_ids = np.sort(train_df["movie_id"].unique())

    user2idx = {u: i for i, u in enumerate(user_ids)}
    movie2idx = {m: i for i, m in enumerate(movie_ids)}

    return user2idx, movie2idx

def encode_df(df, user2idx, movie2idx):
    x_user = df["user_id"].map(user2idx).to_numpy(dtype=np.int64)
    x_movie = df["movie_id"].map(movie2idx).to_numpy(dtype=np.int64)
    y = df["rating"].to_numpy(dtype=np.float32)

```

```

    return x_user, x_movie, y

user2idx, movie2idx = build_id_maps(train_df)

x_user_train, x_movie_train, y_train = encode_df(train_df, user2idx,
movie2idx)
x_user_val, x_movie_val, y_val = encode_df(val_df, user2idx, movie2idx)
x_user_test, x_movie_test, y_test = encode_df(test_df, user2idx, movie2idx)

def make_loader(x_user, x_movie, y, batch_size=BATCH_SIZE, shuffle=False):
    dataset = TensorDataset(
        torch.from_numpy(x_user).long(),
        torch.from_numpy(x_movie).long(),
        torch.from_numpy(y).float()
    )
    return DataLoader(dataset, batch_size=batch_size, shuffle=shuffle,
num_workers=0)

train_loader = make_loader(x_user_train, x_movie_train, y_train,
batch_size=BATCH_SIZE, shuffle=True)
val_loader = make_loader(x_user_val, x_movie_val, y_val,
batch_size=BATCH_SIZE, shuffle=False)
test_loader = make_loader(x_user_test, x_movie_test, y_test,
batch_size=BATCH_SIZE, shuffle=False)

class NeuralCF(nn.Module):
    def __init__(self, n_users, n_movies, embed_dim=EMBED_DIM,
hidden_units=HIDDEN_UNITS, dropout_rate=DROPOUT_RATE):
        super().__init__()

        self.user_embedding = nn.Embedding(n_users, embed_dim)
        self.movie_embedding = nn.Embedding(n_movies, embed_dim)

        layers_list = []
        input_dim = embed_dim * 2

        for units in hidden_units:
            layers_list.append(nn.Linear(input_dim, units))
            layers_list.append(nn.ReLU())
            if dropout_rate > 0:
                layers_list.append(nn.Dropout(dropout_rate))
            input_dim = units

        layers_list.append(nn.Linear(input_dim, 1))
        self.mlp = nn.Sequential(*layers_list)

    def forward(self, user_idx, movie_idx):
        user_vec = self.user_embedding(user_idx)
        movie_vec = self.movie_embedding(movie_idx)
        x = torch.cat([user_vec, movie_vec], dim=1)
        out = self.mlp(x).squeeze(1)
        return out

def clip_predictions(preds, low=1.0, high=5.0):

```

```

    return np.clip(preds, low, high)

def rmse(y_true, y_pred):
    return np.sqrt(np.mean((y_true - y_pred) ** 2))

@torch.no_grad()
def evaluate(model, loader, device=DEVICE):
    model.eval()

    all_preds = []
    all_true = []

    for user_idx, movie_idx, rating in loader:
        user_idx = user_idx.to(device)
        movie_idx = movie_idx.to(device)
        rating = rating.to(device)

        preds = model(user_idx, movie_idx)

        preds_np = preds.cpu().numpy()
        rating_np = rating.cpu().numpy()

        preds_np = clip_predictions(preds_np, 1.0, 5.0)

        all_preds.append(preds_np)
        all_true.append(rating_np)

    y_pred = np.concatenate(all_preds)
    y_true = np.concatenate(all_true)

    return rmse(y_true, y_pred), y_true, y_pred

def train_model(model, train_loader, val_loader, epochs=EPOCHS,
learning_rate=LEARNING_RATE, patience=PATIENCE, device=DEVICE):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    history = {"train_rmse": [], "val_rmse": []}

    best_val_rmse = float("inf")
    best_state = None
    bad_epochs = 0

    for epoch in range(1, epochs + 1):
        model.train()

        train_preds = []
        train_true = []

        for user_idx, movie_idx, rating in train_loader:
            user_idx = user_idx.to(device)
            movie_idx = movie_idx.to(device)
            rating = rating.to(device)

            optimizer.zero_grad()
            preds = model(user_idx, movie_idx)

```

```

        loss = criterion(preds, rating)
        loss.backward()
        optimizer.step()

        train_preds.append(preds.detach().cpu().numpy())
        train_true.append(rating.detach().cpu().numpy())

    train_preds = np.concatenate(train_preds)
    train_true = np.concatenate(train_true)
    train_preds = clip_predictions(train_preds, 1.0, 5.0)
    train_rmse = rmse(train_true, train_preds)

    val_rmse, _, _ = evaluate(model, val_loader, device=device)

    history["train_rmse"].append(train_rmse)
    history["val_rmse"].append(val_rmse)

    if val_rmse < best_val_rmse:
        best_val_rmse = val_rmse
        best_state = copy.deepcopy(model.state_dict())
        bad_epochs = 0
    else:
        bad_epochs += 1
        if bad_epochs >= patience:
            break

    if best_state is not None:
        model.load_state_dict(best_state)

    return model, history

gc.collect()
set_seed(SEED)

model = NeuralCF(
    n_users=len(user2idx),
    n_movies=len(movie2idx),
    embed_dim=EMBED_DIM,
    hidden_units=HIDDEN_UNITS,
    dropout_rate=DROPOUT_RATE
).to(DEVICE)

model, history = train_model(
    model,
    train_loader,
    val_loader,
    epochs=EPOCHS,
    learning_rate=LEARNING_RATE,
    patience=PATIENCE,
    device=DEVICE
)

val_rmse, _, _ = evaluate(model, val_loader, device=DEVICE)
test_rmse, y_test_final, y_pred_final = evaluate(model, test_loader,
device=DEVICE)

```


Bayesian PMF in R

The following code contains the training implementation of the Bayesian PMF model used in our comparison.

```
# Bayesian PMF training implementation in R

set.seed(42)

# 1. Load data
train_path <- "train_ratings.csv"
val_path    <- "val_ratings.csv"
test_path   <- "test_ratings.csv"

train_df <- read.csv(train_path)
val_df    <- read.csv(val_path)
test_df   <- read.csv(test_path)

# 2. Build mappings
all_user_ids <- unique(c(train_df$user_id, val_df$user_id,
test_df$user_id))
all_movie_ids <- unique(c(train_df$movie_id, val_df$movie_id,
test_df$movie_id))

n_users <- length(all_user_ids)
n_items <- length(all_movie_ids)

user2idx <- setNames(seq_along(all_user_ids), all_user_ids)
movie2idx <- setNames(seq_along(all_movie_ids), all_movie_ids)

map_df <- function(df) {
  df$u_idx <- as.integer(user2idx[as.character(df$user_id)])
  df$i_idx <- as.integer(movie2idx[as.character(df$movie_id)])
  df
}

train_df <- map_df(train_df)
val_df    <- map_df(val_df)
test_df   <- map_df(test_df)

# 3. Helpers
clip_rating <- function(x) {
  pmin(5, pmax(1, x))
}

rmse <- function(actual, pred) {
  sqrt(mean((actual - pred)^2))
}

# 4. Train Bayesian-style PMF model
fit_bayesian_style_pmf <- function(train_df, val_df, test_df,
                                   n_users, n_items,
                                   k = 20, epochs = 10,
                                   lr = 0.01, reg = 0.02, seed = 42) {
  set.seed(seed)

  global_mean <- mean(train_df$rating)
```

```

P <- matrix(rnorm(n_users * k, mean = 0, sd = 0.1), nrow = n_users, ncol =
k)
Q <- matrix(rnorm(n_items * k, mean = 0, sd = 0.1), nrow = n_items, ncol =
k)

bu <- rep(0, n_users)
bi <- rep(0, n_items)

history <- data.frame(
  epoch = integer(),
  train_rmse = numeric(),
  val_rmse = numeric()
)

for (ep in 1:epochs) {
  order_idx <- sample(1:nrow(train_df))

  for (idx in order_idx) {
    u <- train_df$u_idx[idx]
    i <- train_df$i_idx[idx]
    r <- train_df$rating[idx]

    pred <- global_mean + bu[u] + bi[i] + sum(P[u, ] * Q[i, ])
    err <- r - pred

    bu[u] <- bu[u] + lr * (err - reg * bu[u])
    bi[i] <- bi[i] + lr * (err - reg * bi[i])

    pu_old <- P[u, ]
    qi_old <- Q[i, ]

    P[u, ] <- P[u, ] + lr * (err * qi_old - reg * pu_old)
    Q[i, ] <- Q[i, ] + lr * (err * pu_old - reg * qi_old)
  }

  train_pred <- global_mean +
    bu[train_df$u_idx] +
    bi[train_df$i_idx] +
    rowSums(P[train_df$u_idx, , drop = FALSE] * Q[train_df$i_idx, , drop =
FALSE])

  val_pred <- global_mean +
    bu[val_df$u_idx] +
    bi[val_df$i_idx] +
    rowSums(P[val_df$u_idx, , drop = FALSE] * Q[val_df$i_idx, , drop =
FALSE])

  train_pred <- clip_rating(train_pred)
  val_pred <- clip_rating(val_pred)

  train_rmse_val <- rmse(train_df$rating, train_pred)
  val_rmse_val <- rmse(val_df$rating, val_pred)

  history <- rbind(
    history,
    data.frame(epoch = ep, train_rmse = train_rmse_val, val_rmse =
val_rmse_val)
  )
}

```

```

}

test_pred <- global_mean +
  bu[test_df$u_idx] +
  bi[test_df$i_idx] +
  rowSums(P[test_df$u_idx, , drop = FALSE] * Q[test_df$i_idx, , drop =
FALSE])

test_pred <- clip_rating(test_pred)
test_rmse_val <- rmse(test_df$rating, test_pred)

model <- list(
  global_mean = global_mean,
  bu = bu,
  bi = bi,
  P = P,
  Q = Q,
  history = history,
  test_rmse = test_rmse_val,
  k = k,
  epochs = epochs,
  lr = lr,
  reg = reg,
  seed = seed
)

return(model)
}

# 5. Fit final model
model <- fit_bayesian_style_pmf(
  train_df = train_df,
  val_df = val_df,
  test_df = test_df,
  n_users = n_users,
  n_items = n_items,
  k = 20,
  epochs = 10,
  lr = 0.01,
  reg = 0.02,
  seed = 42
)

```